Compiler Design Lab.

Compiler Design Lab.

- **Lexical Analysis**: Dividing the input into meaningful units. For a C program the units are *variables*, *constants*, *keywords*, *operators*, *punctuation* etc. These units also called as tokens. (we will use **Flex**)
  - **Flex: A fast Lexical Analyzer Generator**

- **Parsing**: Involves finding the relationship between input tokens. For a C program, one needs to identify valid expressions, statements, blocks, procedures etc. (Use **Bison**)

Flex

- Flex is a tool for generating scanners.
- Flex source is a table of regular expressions and corresponding program fragments.
- Generates *lex.yy.c* which defines a routine *yylex()*

Flex



.l file ⇒ lex ⇒ lex.yy.c

lex.yy.c and other files ⇒ gcc ⇒ lexical analyzer

input stream ⇒ lexical analyzer ⇒ actions taken when rules applied

Flex specifications



{ definitions }

%%

{ rules }

%%
{ user subroutines }

Simple Example


```
%%
.   {printf("Character");}
%%

int yywrap()
{
}
main()
{
yylex();
printf("Hello World");
}
```

Definitions

- The *definitions section contains declarations of simple name definitions to simplify the scanner specification*, and declarations of *start conditions*,
- Name definitions have the form:

  name definition

  DIGIT [0-9]

  ID [a-z][a-z0-9]*

Rules

- The *rules section of the flex input contains a series of rules of the form:*

  pattern action

  where the pattern must be **unintended** and the action must begin on the **same line**.

Flex actions



Actions are C source fragments. If it is compound, or takes more than one line, enclose with braces (’{’ ’}’).

Example rules:

```
[a-z]+              printf("found word\n");
[A-Z][a-z]*         { printf("found capitalized
                            word:\n");
                    printf(" ’%s’\n",yytext);
                  }
```

Flex Pattern Matching



Metacharacter	Matches
.	any character except newline
\n	newline
*	zero or more copies of the preceding expression
+	one or more copies of the preceding expression
?	zero or one copy of the preceding expression
^	beginning of line
$	end of line
a|b	a or b
(ab)+	one or more copies of ab (grouping)
"a+b"	literal "a+b" (C escapes still work)
[ ]	Character class

Flex regular expressions

s	string s literally
\c	character c ; literally, where c would normally be a lex operator
[s]	character class
[~s]	characters not in character class
[s-t]	range of characters
s?	s occurs zero or one time

Flex regular expressions (Cont.)



s*	zero or more occurrences of s
s+	one or more occurrences of s
r|s	r or s
(s)	grouping
$	end of line
s/r	s iff followed by r (not recommended) (r is*NOT*consumed)
s{m,n}	m through n occurrences of s

Examples of regular expressions in flex

a*	zero or more a's
.*	zero or more of any character except newline
.+	one or more characters
[a-z]	a lowercase letter
[a-zA-Z]	any alphabetic letter
[^a-zA-Z]	any non-alphabetic character
a.b	a followed by any character followed by b
rs|tu	rs or tu

Examples of regular expressions in flex



a(b|c)d	abd or acd
^start	beginning of line with then the literal characters start
END$	the characters END followed by an end-of-line.

Write a program in flex

1. TO DETECT POSITIVE AND NEGATIVE INTEGER
2. TO DETECT FLOATING POINT NUMBER

Example : detect positive and negative integer


```
%%
[+]?[0-9]+          {printf("positive integer\n");}
[-]?[0-9]+          {printf("negative integer\n");}
%%
int yywrap()
{
    return 1;
}
int main()
{
printf("positive and negative integer recognition\n");
    yylex();
return 0;
}
```

Thank you

Todays Work

- Write a program in Flex to detect
  - an ID of the form “Start with capital letter then small letters and special characters
  - The key words of standard C program